

Introducción a Arduino Uno

Arduino Uno es una de las tarjetas de desarrollo más populares en el mundo de la electrónica. Es una placa electrónica basada en un **microcontrolador ATmega328P**, que permite a los usuarios programar y controlar dispositivos electrónicos de una manera sencilla y accesible. Con una amplia variedad de pines de entrada y salida, *Arduino Uno puede ser utilizado para crear una gran cantidad de proyectos, desde robots y sistemas de control automatizados hasta dispositivos de monitoreo y medición.* Además, gracias a su amplia comunidad de usuarios, la plataforma Arduino **cuenta con una gran cantidad de recursos, tutoriales y proyectos en línea** para ayudar a los usuarios a comenzar a utilizarla de manera rápida y fácil.

Componentes Básicos

A continuación, te describo las partes principales de la tarjeta y sus componentes:

1. **Microcontrolador ATmega328P:** Este es el cerebro de la tarjeta Arduino Uno, el cual contiene la unidad central de procesamiento (CPU), la memoria de programa, la memoria de datos y otros circuitos periféricos necesarios para su funcionamiento.
2. **Cristal oscilador:** Este componente proporciona al microcontrolador una señal de reloj precisa para que pueda ejecutar el código de manera sincronizada.
3. **Puertos digitales:** La tarjeta Arduino Uno cuenta con 14 pines digitales que se pueden utilizar como entradas o salidas. Cada uno de estos pines puede ser programado para ser utilizado como entrada digital, salida digital, o como entrada analógica.
4. **Puertos analógicos:** La tarjeta Arduino Uno cuenta con 6 pines analógicos que se pueden utilizar como entradas analógicas. Cada uno de estos pines puede ser programado para medir señales de voltaje analógicas.
5. **Puerto USB:** La tarjeta Arduino Uno se conecta a un ordenador mediante un puerto USB para poder programarla y comunicarse con ella.
6. **Conector de alimentación:** La tarjeta Arduino Uno se puede alimentar a través de este conector utilizando una fuente de alimentación externa de entre 7 y 12 voltios.

7. **Conector ICSP:** Este conector se utiliza para programar la tarjeta Arduino Uno utilizando un programador externo.

Los casos de uso de la tarjeta Arduino Uno son variados y van desde proyectos simples para principiantes hasta proyectos más avanzados para profesionales.

Algunos ejemplos de proyectos que se pueden realizar con la tarjeta Arduino Uno son:

- Control de motores
- Sistemas de seguridad
- Robots
- Sensores de temperatura y humedad
- Sistemas de iluminación automatizados
- Medidores de distancia

Funciones Básicas

Algunas de las funciones básicas más utilizadas en la programación de Arduino, junto con sus conceptos, descripciones, ejemplos y casos de uso:

1. **setup():** Esta función se ejecuta una sola vez al inicio del programa, y se utiliza para inicializar los pines y los periféricos de la tarjeta Arduino. La sintaxis de la función es la siguiente:

```
void setup() {  
  // Código de inicialización  
}
```

Un ejemplo de uso de esta función sería el siguiente:

```
void setup() {  
  pinMode(13, OUTPUT); // Configura el pin 13 como salida  
}
```

2. **loop():** Esta función se ejecuta continuamente en un ciclo infinito después de que la función **setup()** ha sido ejecutada. Se utiliza para ejecutar el código principal del programa. La sintaxis de la función es la siguiente:

```
void loop() {  
  // Código principal  
}
```

Un ejemplo de uso de esta función sería el siguiente:

```
void loop() {  
  digitalWrite(13, HIGH); // Enciende el LED conectado al pin 13  
  delay(1000); // Espera un segundo  
  digitalWrite(13, LOW); // Apaga el LED conectado al pin 13  
  delay(1000); // Espera un segundo  
}
```

Este código encenderá y apagará el LED conectado al pin 13 de la tarjeta Arduino a intervalos de un segundo.

3. **digitalWrite()**: Esta función se utiliza para enviar una señal de voltaje a un pin digital de la tarjeta Arduino. La sintaxis de la función es la siguiente:

```
digitalWrite(pin, value);
```

Donde **pin** es el número del pin digital y **value** es el valor que se desea enviar al pin (HIGH o LOW). Un ejemplo de uso de esta función sería el siguiente:

```
digitalWrite(13, HIGH); // Enciende el LED conectado al pin 13
```

4. **analogRead()**: Esta función se utiliza para leer el valor analógico de un pin analógico de la tarjeta Arduino. La sintaxis de la función es la siguiente:

```
analogRead(pin);
```

Donde **pin** es el número del pin analógico que se desea leer. Un ejemplo de uso de esta función sería el siguiente:

```
int valor = analogRead(A0); // Lee el valor analógico del pin A0
```

5. `delay()`: Esta función se utiliza para hacer una pausa en la ejecución del programa por un período de tiempo determinado. La sintaxis de la función es la siguiente:

```
delay(time);
```

Donde `time` es el tiempo en milisegundos que se desea esperar. Un ejemplo de uso de esta función sería el siguiente:

```
delay(1000); // Espera un segundo
```

Algunos casos de uso de estas funciones podrían ser:

- Utilizar `setup()` para configurar los pines de la tarjeta Arduino y `loop()` para ejecutar el código principal del programa.
- Utilizar `digitalWrite()` para encender o apagar un LED conectado a un pin digital de la tarjeta Arduino.
- Utilizar `analogRead()` para medir el valor analógico de un sensor conectado a un pin analógico de la tarjeta Arduino.
- Utilizar `delay()` para hacer una pausa en la ejecución del programa.

Código C/C++ en Arduino Uno

Declaración de Variables:

Las variables son elementos fundamentales en la programación y permiten almacenar valores temporales o permanentes que pueden ser utilizados en el programa. En Arduino Uno, se pueden declarar variables de distintos tipos, como enteros (`int`), caracteres (`char`), flotantes (`float`), booleanos (`bool`), entre otros.

Para declarar una variable en Arduino Uno, es necesario especificar el tipo de variable que se va a utilizar, seguido del nombre que se le va a asignar. Por ejemplo, para declarar una variable entera llamada `valor`, se puede escribir:

```
int valor;
```

Después de declarar la variable, se puede asignar un valor inicial si se desea. Por ejemplo:

```
int valor = 5;
```

También se pueden declarar múltiples variables del mismo tipo en una misma línea, separando cada una por una coma. Por ejemplo:

```
int valor1, valor2, valor3;
```

Es importante destacar que el nombre de la variable debe ser único dentro del programa y no debe contener caracteres especiales ni espacios en blanco.

Condicionales:

En Arduino Uno, al igual que en otros lenguajes de programación, el comando `if` se utiliza para realizar una comparación y tomar una acción específica en función del resultado de esa comparación.

La sintaxis básica de un `if` en Arduino Uno es la siguiente:

```
if (condición) {  
    // Acción a realizar si se cumple la condición  
}
```

En este caso, `condición` es la comparación que se quiere realizar, y `Acción a realizar si se cumple la condición` es el código que se ejecutará si la condición es verdadera. Si la condición es falsa, el código dentro del `if` será ignorado y el programa continuará con la siguiente línea de código.

Por ejemplo, si queremos comparar si el valor de una variable `x` es mayor que 5 y, en caso afirmativo, encender un LED conectado al pin 13, se podría escribir el siguiente código:

```
int x = 7;  
  
if (x > 5) {
```

```
digitalWrite(13, HIGH);  
}
```

En este caso, la condición `x > 5` se cumple ya que el valor de `x` es 7, por lo que se ejecutará la línea `digitalWrite(13, HIGH);`, que enciende el LED conectado al pin 13.

Tanto `if` como `switch` son estructuras de control de flujo que permiten ejecutar diferentes acciones dependiendo del valor de una expresión o condición. Si se necesita ejecutar diferentes acciones en función de varias posibles condiciones, se puede utilizar un condicional múltiple.

En el caso de `if`, se pueden encadenar varias estructuras `if` utilizando la sintaxis `else if`, de la siguiente manera:

```
int x = 3;  
  
if (x == 1) {  
    // Acción a realizar si x es igual a 1  
} else if (x == 2) {  
    // Acción a realizar si x es igual a 2  
} else if (x == 3) {  
    // Acción a realizar si x es igual a 3  
} else {  
    // Acción a realizar si x no es igual a ninguno de los valores anteriores  
}
```

En este caso, se están comparando diferentes valores de `x` en cada estructura `if` utilizando el operador de igualdad `==`. Si ninguna de las condiciones se cumple, se ejecutará la acción dentro del bloque `else`.

```
int sensorValue = analogRead(A0);  
  
if (sensorValue < 100) {  
    digitalWrite(ledPin, HIGH);  
} else if (sensorValue < 200) {  
    digitalWrite(ledPin, LOW);  
    delay(50);  
    digitalWrite(ledPin, HIGH);  
    delay(50);  
} else if (sensorValue < 300) {  
    digitalWrite(ledPin, LOW);  
    delay(50);  
    digitalWrite(ledPin, HIGH);  
    delay(50);  
    digitalWrite(ledPin, LOW);  
}
```

```
    delay(50);
    digitalWrite(ledPin, HIGH);
    delay(50);
} else {
    digitalWrite(ledPin, LOW);
}
```

En este ejemplo, se utiliza un sensor conectado al pin analógico A0 para leer un valor numérico entre 0 y 1023. Dependiendo del valor leído, se ejecutan diferentes acciones mediante el uso de estructuras `if` con `else if`. Si el valor es menor a 100, se enciende un LED conectado al pin digital correspondiente. Si es mayor o igual a 100 y menor a 200, se parpadea el LED. Si es mayor o igual a 200 y menor a 300, se realiza un patrón de parpadeo diferente. Si es mayor o igual a 300, el LED se apaga.

Por otro lado, en el caso de `switch`, se puede utilizar la siguiente sintaxis para realizar un condicional múltiple:

```
int x = 2;

switch (x) {
    case 1:
        // Acción a realizar si x es igual a 1
        break;
    case 2:
        // Acción a realizar si x es igual a 2
        break;
    case 3:
        // Acción a realizar si x es igual a 3
        break;
    default:
        // Acción a realizar si x no es igual a ninguno de los valores anteriores
        break;
}
```

En este caso, se utiliza la palabra clave `switch` seguida de la expresión a comparar. Dentro del bloque de código del `switch`, se utilizan diferentes casos (`case`) para cada posible valor de la expresión. Si el valor coincide con alguno de los casos, se ejecutará la acción correspondiente. Si ninguno de los casos se cumple, se ejecutará el bloque `default`.

```
int buttonState = digitalRead(buttonPin);

switch (buttonState) {
    case HIGH:
        digitalWrite(ledPin, HIGH);
```

```
    break;
case LOW:
    digitalWrite(ledPin, LOW);
    break;
default:
    break;
}
```

En este ejemplo, se utiliza un botón conectado al pin digital correspondiente para leer su estado (alto o bajo). Dependiendo del estado leído, se enciende o apaga un LED conectado al pin digital correspondiente mediante el uso de una estructura **switch**. Si el estado es alto, el LED se enciende. Si el estado es bajo, el LED se apaga. Si no se cumple ninguna de las condiciones anteriores, no se ejecuta ninguna acción adicional.

Es importante recordar que en ambos casos, se deben definir previamente los pines a utilizar y los valores límite para cada condición.

También es algo a tomar en cuenta que en ambos casos, se puede utilizar cualquiera de las estructuras según la preferencia del programador o en función de la complejidad del condicional. Además, es necesario utilizar la sintaxis adecuada para cada estructura para evitar errores de compilación.

Ciclos:

```
int ledPin = 13;

void setup() {
    pinMode(ledPin, OUTPUT);
}

void loop() {
    for (int i = 0; i < 5; i++) {
        digitalWrite(ledPin, HIGH);
        delay(100);
        digitalWrite(ledPin, LOW);
        delay(100);
    }
    delay(1000);
}
```

En este ejemplo, se utiliza una estructura **for** para repetir cinco veces un patrón de parpadeo de un LED conectado al pin digital 13. Se utiliza una variable **i** para contar las iteraciones y un **delay** para esperar un tiempo entre cada parpadeo. Después de cada ciclo de cinco parpadeos, se espera un segundo antes de comenzar de nuevo.

Ejemplo con `while`:

```
int buttonPin = 2;
int ledPin = 13;
int buttonState;

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  while (digitalRead(buttonPin) == HIGH) {
    digitalWrite(ledPin, HIGH);
  }
  digitalWrite(ledPin, LOW);
}
```

En este ejemplo, se utiliza una estructura `while` para mantener encendido un LED conectado al pin digital 13 mientras se mantenga presionado un botón conectado al pin digital 2. Se utiliza una variable `buttonState` para leer el estado del botón y una función `digitalRead()` para leer su estado en cada iteración del bucle. Si el estado es alto, se enciende el LED, y si es bajo, se apaga.

Ejemplo con `do while`:

```
int sensorPin = A0;
int threshold = 500;
int sensorValue;

void setup() {
  Serial.begin(9600);
}

void loop() {
  do {
    sensorValue = analogRead(sensorPin);
    Serial.println(sensorValue);
    delay(100);
  } while (sensorValue < threshold);
  Serial.println("Threshold exceeded");
  delay(1000);
}
```

En este ejemplo, se utiliza una estructura `do while` para leer continuamente un sensor conectado al pin analógico A0 y mostrar su valor en el Monitor Serial de Arduino. Si el valor leído es menor a un umbral de 500, se sigue repitiendo el ciclo.

Cuando el valor leído supera el umbral, se muestra un mensaje indicando que se ha superado el umbral y se espera un segundo antes de comenzar de nuevo.

Es importante recordar que en cada caso, se deben definir previamente los pines y variables a utilizar y los límites para cada condición.

Anexos

Tabla de resumen de las partes principales de la tarjeta Arduino Uno y su funcionalidad:

Parte/Componente	Funcionalidad
Microcontrolador ATmega328P	Es el cerebro de la tarjeta y contiene la CPU, la memoria de programa y la memoria de datos.
Cristal oscilador	Proporciona una señal de reloj precisa al microcontrolador.
Puertos digitales	Se utilizan como entradas o salidas para señales digitales.
Puertos analógicos	Se utilizan como entradas analógicas para medir señales de voltaje.
Puerto USB	Se utiliza para programar y comunicarse con la tarjeta.
Conector de alimentación	Se utiliza para alimentar la tarjeta con una fuente de alimentación externa.
Conector ICSP	Se utiliza para programar la tarjeta utilizando un programador externo.

Tabla de resumen de las funciones más usadas para programar la arduino uno

Función	Concepto	Descripción	Ejemplo de uso	Casos de uso
setup()	Función de inicialización	Se ejecuta una sola vez al inicio del programa para inicializar los	pinMode(13, OUTPUT);	Configurar los pines de la tarjeta Arduino y los

Función	Concepto	Descripción	Ejemplo de uso	Casos de uso
		pinos y periféricos de la tarjeta Arduino.		periféricos necesarios para el programa.
<code>loop()</code>	Función de ciclo principal	Se ejecuta continuamente en un ciclo infinito después de que la función <code>setup()</code> ha sido ejecutada. Se utiliza para ejecutar el código principal del programa.	<code>digitalWrite(13, HIGH);</code>	Ejecutar el código principal del programa en un ciclo infinito.
<code>digitalWrite()</code>	Envío de señal de voltaje	Se utiliza para enviar una señal de voltaje a un pin digital de la tarjeta Arduino.	<code>digitalWrite(13, HIGH);</code>	Encender o apagar un LED conectado a un pin digital de la tarjeta Arduino.
<code>analogRead()</code>	Lectura de valores analógicos	Se utiliza para leer el valor analógico de un pin analógico de la tarjeta Arduino.	<code>int valor = analogRead(A0);</code>	Medir el valor analógico de un sensor conectado a un pin analógico

Función	Concepto	Descripción	Ejemplo de uso	Casos de uso
				de la tarjeta Arduino.
<code>delay()</code>	Pausa en la ejecución del programa	Se utiliza para hacer una pausa en la ejecución del programa por un período de tiempo determinado.	<code>delay(1000);</code>	Hacer una pausa en la ejecución del programa para dar tiempo a otros procesos o para realizar acciones específicas.